

Cross Segmentos - Linha Datasul - MLA - Criação de workflows no Fluig para aprovação de documentos padrões ou específicos

 Tempo aproximado para leitura: **00:17:55 min**

Dúvida

Com funciona a criação de **workflows** no **Fluig** para aprovação de documentos padrões ou específicos?

Ambiente

Cross Segmentos - TOTVS Backoffice (Linha Datasul) - Aprovação Processo Logístico (MLA) - Versão 12

Solução

Para criar um **workflow** no **Fluig** é preciso ter o **TDS - TOTVS Developer Studio** instalado e devidamente configurado. Para mais informações acesse: TDS Versão 11.3.

Há um projeto de exemplo que deve ser usado como base para a criação de novos **workflows** para documentos padrões ou específicos. Está disponível no diretório de expedição dos arquivos **progress**, seguindo a estrutura: **ems2/fluig/process/wfmla000.zip**

Seguindo este projeto, algumas funções padrões estarão disponíveis, são elas:

1. Criação da pendência no **Fluig** quando a pendência for criada no **ERP**, se estiver parametrizado corretamente;
2. Visualização do detalhe da pendência;
3. Visualização do histórico de aprovações;
4. Busca das informações no Sistema, da pendência a ser aprovada/rejeitada;
5. Ações da aprovação - formulário: **Aprovar/Reprovar, Narrativa e Código de Rejeição**, de acordo com cadastro no Sistema;
6. Validações de aprovação: Limite, tipo, permissão, etc;
7. Bloqueio na inicialização do **workflow** diretamente pelo **Fluig**;
8. Bloqueio no cancelamento do **workflow** diretamente pelo **Fluig**.

Para criar um novo **workflow** deve-se seguir as informações abaixo:

1. Copie este projeto, renomeie os arquivos seguindo a nomenclatura conforme o código do documento: **WFMLA + código do documento com 3 dígitos**.

Exemplos:

Para o documento 1 será **WFMLA001**

Para o documento 501 será **WFMLA501**

Os arquivos que precisam ser renomeados são: **wfmla000.html**, **wfmla000.js**, **wfmla000.css**, **wfmla000_en_US.properties**, **wfmla000_es.properties**, **wfmla000_pt_BR.properties**, **wfmla000.afterTaskComplete.js**, **wfmla000.afterTaskCreate.js**, **wfmla000.beforeCancelProcess.js**, **wfmla000.process**

2. Após renomear os arquivos, importe o projeto no **TDS**.
3. Altere os arquivos que houver necessidade, conforme documentações e orientações abaixo.
4. Exporte o mecanismo **PorGestorProcesso.js** para o servidor.
5. Exporte o **dataset gestorProcesso.js**.
6. Exporte o formulário para o servidor.
7. Abra as propriedades do diagrama, e na aba **Formulário** selecione o formulário exportado no passo 5.
8. Ainda na aba **Formulário**, selecione e informe a descrição dos campos descritores, para mais informações consultar a documentação do **workflow**.
9. Por fim, exporte o diagrama para o servidor.

Documentação dos arquivos do Projeto:

1. **HTML:** O arquivo **.html** do formulário deve seguir a estrutura mostrada no **anexo wfmla000.html** para que as funcionalidades padrões funcionem. Como o **Detalhe da pendência** e **Histórico de Aprovação**.

2. Javascript

- o **datasul-11.5.X-SNAPSHOT.js**: arquivo com as funções padrões de conexão com o Sistema. **Este arquivo não deve ser alterado.**
- o **ttDados.js**: arquivo com as funções de conversão dos dados trafegados entre o Sistema **Datasul** e o **Fluig**. **Este arquivo não deve ser alterado.**
- o **wfmla000.js**: funcionalidades do formulário. Abaixo estão descritas as principais funções deste arquivo:
 - **getTemplates() - Altere para carregar os templates necessários**
Função criada para disparar a busca do **template** do **HTML**, fica escondido dentro da **div** com **id 'templates'**.
 - Inicializa as variáveis de **template**.
 - Dispara a função **setTemplate**, passando o **id** da **div** onde está o **template**, e a variável que irá armazenar este conteúdo.

Os templates padrões (i18n_sim, i18n_nao, i18n_show, i18n_hide, templateNaoHa, templateDetalhePendencia, templateHistorico e templateChaveDoc) não devem ser alterados.

getTemplates()

```
function getTemplates(){  
    /* Templates padrões */  
    /*Tradução dos valores lógicos Sim/Não*/
```

```

i18n_sim = $("#i18n_sim").html();
i18n_nao = $("#i18n_nao").html();

templateNaoHa = null;
setTemplate("template_nao_ha", "templateNaoHa");
templateDetalhePendencia = null;
setTemplate("template_detalhe_pendencia", "templateDetalhePendencia");
templateHistorico = null;
setTemplate("template_historico", "templateHistorico");
templateChaveDoc = null;
setTemplate("template_chave_doc", "templateChaveDoc");

/* Informe aqui os templates específicos ou customizados */
templateCabecalho = null;
setTemplate("template_cabecalho", "templateCabecalho");
templateItens = null;
setTemplate("template_itens", "templateItens");
}

```

- **setTemplate(id_template_html, nome_variavel) - Não deve ser alterado.**

1. **id_template_html:** id da **div html** que armazena o **template** no arquivo **html**.

2. **nome_variavel:** nome da variável que irá armazenar o conteúdo do **template**.

Esta função busca o **template** no arquivo **HTML** e salva em uma variável para depois fazer a renderização dos dados nas marcações feitas no **template** - ver função **layout()**.

setTemplate(nomeTemplate,variavel)

```

function setTemplate(nomeTemplate,variavel){
    template = document.getElementById(nomeTemplate).innerHTML;
    template = template.replace(/(\r\n|\n|\r)/gm, ""); //remove os 'enter'
    template = template.replace(new RegExp("'", "g"), "\\'"); //escapa as
    aspas simples '
    eval(variavel + " = '" + template + "';");
}

```

- **montarTela() - Altere de acordo com a sua estrutura de dados em tela: exibição simples, repetição, etc...**

Esta função é responsável pelas chamadas da função **layout()**, que mostra as informações na tela. É nesta função que devem ser incluídas as informações específicas ou customizadas. Abaixo estão listados os três tipos de exibição de dados disponíveis para utilização:

1. **Exibição simples, sem repetição e sem restrição:**

Executa apenas uma vez a função **layout**. Usa-se este método quando há um **array** com apenas um registro, por exemplo um cabeçalho de pedido de compra.

Ex: **layout(templateCabecalho,"tt_pedido","cabecalho");**

template

```
<div id="template_cabecalho">
    <div id="cabecalho">
        <div class="row">
            <div class="col-xs-12 col-sm-2 col-md-2
labelField">i18n.translate("l-order")</div>
            <div class="col-xs-12 col-sm-9 col-md-10 valueField">
{{tt_pedido.num_pedido}}</div>
        </div>
        <div class="row">
            <div class="col-xs-12 col-sm-2 col-md-2
labelField">i18n.translate("l-site")</div>
            <div class="col-xs-12 col-sm-9 col-md-10 valueField">
{{tt_pedido.cod_estabel}} - {{tt_pedido.desc_cod_estabel}}</div>
        </div>
        <div class="row">
            <div class="col-xs-12 col-sm-2 col-md-2
labelField">i18n.translate("l-vendor")</div>
            <div class="col-xs-12 col-sm-9 col-md-10 valueField">
{{tt_pedido.cod_emitente}} - {{tt_pedido.desc_cod_emitente}}</div>
        </div>
        <div class="row">
            <div class="col-xs-12 col-sm-2 col-md-2
labelField">i18n.translate("l-company-name")</div>
            <div class="col-xs-12 col-sm-9 col-md-10 valueField">
{{tt_pedido.razao_social}}</div>
        </div>
    </div>
</div>
```

Função layout

```
templateCabecalho = null;
setTemplate("template_cabecalho", "templateCabecalho");
layout(templateCabecalho, "tt_pedido", "cabecalho");
```

- **Resultado em tela:**

Pedido Emergencial - Total



Pedido	240985
Estabelecimento	1 - Estabelec TOTVS
Fornecedor	1 - SASAZAKI
Razão social	SASAZAKI INDÚSTRIA E COMÉRCIO LTDA AÇÃO. 123
Natureza	Compra
Data Pedido	07/07/2016
Frete	Pago
Responsável	super - Super Usuário
Cond. Pagamento	1 - A Vista
Transportador	0 - DESTINATÁRIO
Via transporte	Rodoviário
Comentários	
Valor Total do Pedido	5,00

- **Exibição repetitiva, sem restrição:**

Executa a função **layout** para cada **index** do **array** - cada registro da **temp-table**, sem filtro. Usa-se quando é necessário mostrar todos os registros da **temp-table**, por exemplo exibir todas as ordens de compra.

Ex: `layout(templateOrdens,"tt_ordem_compra","ordens");`

Template Entregas

```
<div id="template_entregas">
  <div class="col-xs-12 col-sm-12 col-md-12 item">
    <div class="row">
      <div class="col-xs-12 col-sm-2 col-md-2
labelField">i18n.translate("l-part-short")</div>
      <div class="col-xs-12 col-sm-2 col-md-2 valueField">
{{tt_prazo_compra.parcela}}</div>
      <div class="col-xs-12 col-sm-2 col-md-2
labelField">i18n.translate("l-delivery-date")</div>
      <div class="col-xs-12 col-sm-2 col-md-2 valueField">
{{tt_prazo_compra.data_entrega}}</div>
      <div class="col-xs-12 col-sm-2 col-md-2
labelField">i18n.translate("l-um")</div>
      <div class="col-xs-12 col-sm-2 col-md-2 valueField">
{{tt_prazo_compra.un}}</div>
    </div>
    <div class="row">
      <div class="col-xs-12 col-sm-2 col-md-2
labelField">i18n.translate("l-quantity")</div>
      <div class="col-xs-12 col-sm-2 col-md-2 valueField">
{{tt_prazo_compra.quantidade}}</div>
      <div class="col-xs-12 col-sm-2 col-md-2
labelField">i18n.translate("l-quantity-balance-short")</div>
      <div class="col-xs-12 col-sm-2 col-md-2 valueField">
{{tt_prazo_compra.quant saldo}}</div>
    </div>
  </div>
</div>
```

```

<div class="col-xs-12 col-sm-2 col-md-2
labelField">i18n.translate("l-quantity-receipt-short")</div>
<div class="col-xs-12 col-sm-2 col-md-2 valueField">
{{tt_prazo_compra.quant_receb}}</div>
</div>
<div class="row">
<div class="col-xs-12 col-sm-2 col-md-2
labelField">i18n.translate("l-return-quantity-short")</div>
<div class="col-xs-12 col-sm-2 col-md-2 valueField">
{{tt_prazo_compra.quant_rejeit}}</div>
</div>
</div>
</div>

```

Função layout

```

for(var i = 0;i<tt_ordem_compra.length;i++){
    camposChave = ['numero_ordem'];
    valoresChave = [tt_ordem_compra[i].numero_ordem];
    layout(templateEntregas,"tt_prazo_compra","entregas"+i,
camposChave,valoresChave);
}

```

Resultado em tela:

Ordens	Entregas	Cotações	Requisições		
Parc	1	Data Entrega	06/08/2016	UM	PC
Quantidade	1,0000	Qtd Saldo	1,0000	Qtde Receb	0,0000
Qtde Devol	0,0000				
Parc	2	Data Entrega	06/08/2016	UM	PC
Quantidade	2,0000	Qtd Saldo	2,0000	Qtde Receb	0,0000
Qtde Devol	0,0000				
Parc	3	Data Entrega	06/08/2016	UM	PC
Quantidade	1,0000	Qtd Saldo	1,0000	Qtde Receb	0,0000
Qtde Devol	0,0000				
Parc	4	Data Entrega	06/08/2016	UM	PC
Quantidade	1,0000	Qtd Saldo	1,0000	Qtde Receb	0,0000
Qtde Devol	0,0000				

- **layout(template, nomeTT, divDestino, camposChave, valoresChave)** - Não deve ser alterado.
 - **template:** template **HTML**.
 - **nomeTT:** nome da **temp-table (array)** que possui os dados que serão mostrados no template.
 - **divDestino:** **ID** da **div** que receberá o **template** renderizado com os dados da **temp-table** nas marcações.
 - **camposChave:** **array** com os nomes dos campos usados no filtro.

- o **valoresChave: array** com os valores dos campos usados no filtro.

Esta função tem como objetivo substituir as marcações do **template** pelos dados da **temp-table**.

- **detalheDocumento() - Pode ser alterado.**

Esta função consome os dados do método **dts_detalheDocumento()** do **displayFields.js** que faz a busca dos dados do documento no Sistema. Neste ponto é executada a **procedure detalheDocumento** da **mlaapi014**. O programa **Progress** chamado deve ter a nomenclatura de acordo com as regras do **MLA**. Para mais informações sobre a criação dos programas **Progress** para documentos customizados, verificar artigo Cross Segmentos - Linha Datasul - MLA - Como criar documentos para o Portal do MLA em HTML ou customizar documentos existentes.

- **detalhePendencia() - Pode ser alterado**

Esta função consome os dados do método **dts_detalhePendencia()** do **displayFields.js** que faz a busca dos dados da pendência no Sistema. Neste ponto é executada a **procedure detalhePendencia** da **mlaapi014** que busca as informações da tabela **mmla-docto-pend-aprovac-det**. Serão retornadas as informações do detalhe da pendência e da chave do documento.

- **historicoDocumento() - Pode ser alterado**

Esta função consome os dados do método **dts_historicoDocumento()** do **displayFields.js** que faz a busca dos dados do histórico do documento no Sistema. Neste ponto é executada a **procedure historicoDocumento** da **mmlaapi014** que retorna as informações do histórico de aprovações do documento.

- **getCodRejeita() - Pode ser alterado**

Esta função consome os dados do método **dts_getCodRejeita()** do **displayFields.js** que faz a busca dos dados dos motivos de rejeição no Sistema. Neste ponto é executada a **procedure getCodRejeita** da **mmlaapi014** que retorna as informações dos motivos de rejeição do documento.

- **ttDados - Não deve ser alterado**

Essa é a variável que armazena os dados trafegados entre o Sistema **Datasul** e o **Fluig**. Ela armazena todas as informações das **temp-tables** do **Progress**.

Assim que o retorno é recebido, é executada a função **transformTT**, onde é feita a lógica de criação das variáveis. As **temp-tables** transformam-se em **arrays**, com uma diferença: troca-se o - **por** _ .

Por exemplo: **tt-it-requisicao - Progress / tt_it_requisicao - HTML**.

Isto serve tanto para o nome quanto para os campos das **temp-tables**.

Pelo **javascript** você consegue acessar os dados vindos do **Progress** da seguinte forma:

Acesso aos dados das temp-tables via javascript

```
for(var i = 0;i<tt_it_requisicao.length;i++){  
    console.info("Item: ", tt_it_requisicao[i].it_codigo);  
    console.info("Requisição: ", tt_it_requisicao[i].nr_requisicao);
```

```
console.info("Seq: ", tt_it_requisicao[i].sequencia);  
console.info("Dt Entrega: ", tt_it_requisicao[i].dt_entrega);  
console.info("Qtd Requisitada: ", tt_it_requisicao[i].qt_requisitada);  
}
```

- **Tradução - Altere para contemplar as literais específicas ou customizadas**

Existem três arquivos referentes a tradução das literais dos formulários, sendo um para cada idioma: **PT, EN e ES**.

O nome dos arquivos deve estar de acordo com o exemplo abaixo:

1. **WFMLA999_en_US.properties**
2. **WFMLA999_es.properties**
3. **WFMLA999_pt_BR.properties**

O **999** deve ser substituído pelo número do documento: **WFMLA001, WFMLA003, WFMLA500, etc.**

A estrutura dos arquivos de tradução seguem a seguinte regra:

- **I-nome-literal**=Valor da literal

Exemplos:

I-accounts-plan=Plano Contas

I-advanced-search=Pesquisa Avançada

I-all=Todas

- **Alteração de programas Progress - Somente para desenvolvimento interno da TOTVS ***

Para desenvolvimento dos documentos padrões do **MLA**, é preciso alterar a **boin784.m01** para que a opção **Integração Fluig** fique habilitada no **MLA0101 - Tipos de documento** e seja considerada na exportação do **MLA0000 - Parâmetros da aprovação**.

***Para desenvolvimento de documentos específicos os parâmetros de integração com o Fluig virão habilitados para serem informados, sendo assim, não precisará ser feita a alteração acima.**

- **Campos Descritores**

Os campos descritores são os campos que aparecem nos filtros customizados na central de tarefas, como mostra a imagem abaixo.

8 - Pedido Emergencial (Total)

Karina Lais

27

tarefas atrasadas

61

tarefas no prazo

88

tarefas pendentes no total

Filtrar nesta página...

Editar filtro

Salvar filtro

Limpar filtro

Exportar

Atualizar

Mais

Pedido	Fornecedor	Razão Social	Data	Cond Pag	Responsável	Natureza	Chave	Empresa	Estab	Valor
240986	SASAZAKI	SASAZAKI INDÚS...	12/07/2016	A Vista	kleger387	Compra	240986	10	1	100,00
240985	SASAZAKI	SASAZAKI INDÚS...	07/07/2016	A Vista	super	Compra	240985	10	1	5,00

Para que estes campos estejam disponíveis no **Fluig** é necessária a seguinte alteração no programa **progress mlaapi014.i**

- **Progress**
 - **Para desenvolvimento interno:** Incluir uma nova cláusula na função **case** incluindo os campos necessários.
 - **Para desenvolvimento de documentos customizados:** Existe um ponto **EPC** para o evento **CamposDescritores** onde é passado como parâmetro o **rowid** da tabela **mla-doc-pend-aprov**. O retorno esperado é uma **string** contendo o nome do campo no **HTML**, ver tópico abaixo, e o valor a ser armazenado, no formato mostrado abaixo:

Campos Descritores

```
<item>
  <item>nome_do_campo_do_formulario_html</item>
  <item>valor_do_campo</item>
</item>
```

- **HTML**

Lembre-se, para que o campo fique disponível nos filtros da central de tarefas, além de alterar o programa **progress** é necessário incluí-lo no **HTML**. Dentro do arquivo **WFMLA000.html** há um **tag <form>**, que possui uma **tag <div class="hidden">**. Dentro desta **div** estão os campos **input**. Deve-se incluir o campo novo no final desta **div**, como o campo **ct_nr_requisicao**, na linha 18 do exemplo abaixo:

Campos Descritores - HTML

```
<form enctype="" name="customForm" action="" class="formee" method="post" style="border:1px solid #ddd;">
    <div class="hidden">
        <br>
        <input type="text" id="nr_transacao" name="nr_transacao" readonly="readonly">
        <br>
        <input type="text" id="usuario" name="usuario" readonly="readonly">
        <br>
```

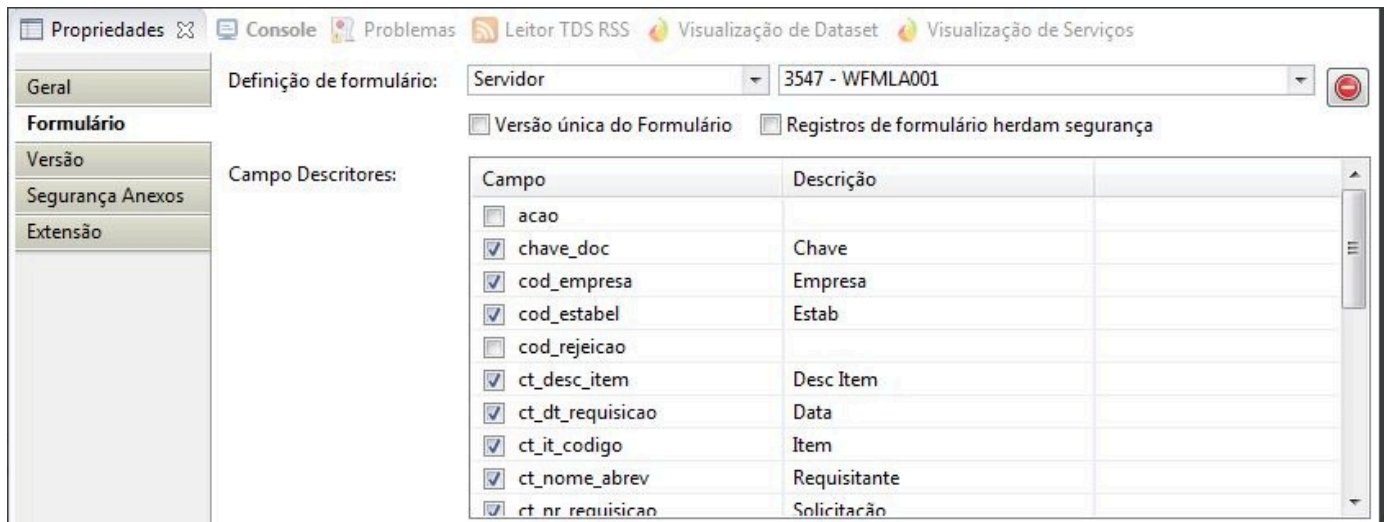
```

<input type="text" id="chave_doc" name="chave_doc" readonly="readonly">
    <br>
<input type="text" id="cod_empresa" name="cod_empresa" readonly="readonly">
    <br>
<input type="text" id="cod_estabel" name="cod_estabel" readonly="readonly">
    <br>
<input type="text" id="valor_doc" name="valor_doc" readonly="readonly">
    <br>
<input type="text" id="validar" name="validar" readonly="readonly">
    <br>
<input type="text" id="validarCampos" name="validarCampos" readonly="readonly"
>
    <br>
<input type="text" id="isProgress" name="isProgress" readonly="readonly">
    <br>
<input type="text" id="prazo" name="prazo" readonly="readonly">
    <br>
<input type="text" id="validarNarrativa" name="validarNarrativa" readonly="rea
donly">
    <br>
<input type="text" id="rejeicao" name="rejeicao" id="rejeicao" readonly="reado
nly">
    <br>
<input type="text" id="rejeicaoPadrao" name="rejeicaoPadrao" readonly="readonl
y">
    <br>
<input type="text" id="usuario_logado" name="usuario_logado" readonly="readonl
y">

    <!-- [ Editável ] Inclua aqui os campos que
poderão ser usados na central de tarefas. Recomenda-se usar ct_ como prefixo
do campo para indicar que é usado na central de tarefas. -->
    <input type="text" id="ct_nr_requisicao" name="ct_
nr_requisicao" readonly="readonly">
    </div>

```

- Por último, é necessário selecionar os campos e preencher suas respectivas descrições na aba **Formulário** nas propriedades do diagrama no **TDS**, como mostra a imagem abaixo:



Para mais informações sobre a seleção dos campos descritores, consultar documentação de implantação de **workflow**.

Formulário Mobile

Para que o formulário esteja disponível no aplicativo do **Fluig**, conforme *link*: Mobile, é preciso criar um arquivo chamado **<Nome_workflow>_ecmmobile.html**. Exemplo: **wfmla001_ecmmobile.html**

Este arquivo é responsável pela exibição dos dados do formulário nos dispositivos móveis. Deve ser desenvolvido levando em consideração o tamanho da tela e a quantidade de informação que será exibida. Uma particularidade deste fonte é que ele não permite o uso de arquivos externos, por isso é preciso que o conteúdo de alguns arquivos sejam copiados para dentro do **HTML**.

São eles: **<nome_workflow>.js**, **datasul-11.5.X-SNAPSHOT.js**, **ttDados.js** e **<nome_workflow>.css**.

Sendo que os arquivos do tipo **Javascript** devem ficar dentro da *tag*: **<script type="text/javascript"> [...] </script>** e os arquivos **CSS** dentro da *tag*: **<style type="text/css"> </style>**

Caso o formulário **desktop** tenha sido desenvolvido usando técnicas de responsividade (**col-xs-***, **col-sm-***, **col-md-*** e **col-lg-***), basta copiar o conteúdo do **HTML** do arquivo **<nome_workflow>.html** e colar no **<nome_workflow>_ecmmobile.html**. Caso contrário é preciso codificar o arquivo **mobile** de acordo com a necessidade para que ele funcione corretamente.

Os dados que serão retornados do Sistema quando o acesso for via dispositivos móveis são os mesmos dados do formulário **desktop**. Cabe ao desenvolvedor avaliar quais informações são relevantes e então deve-se implementá-las no código **HTML** para que sejam exibidas em tela.

Sempre que houver alguma alteração nos arquivos **Javascript e/ou CSS**, a alteração deve ser avaliada e repassada para o arquivo **HTML mobile** se for necessária.

Há um arquivo de exemplo que pode ser consultado em: **ems2/fluig/process/wfmla000.zip** - após descompactar: **WFMLA000/forms/WFMLA000/WFMLA000_ecmmobile..html**.